MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE RUSSIAN FEDERATION

Federal State Autonomous Educational Institution of Higher Education
"Peter the Great St. Petersburg Polytechnic University"

Institute of Computer Science and Cybersecurity

Higher School of Technology and Artificial Intelligence

Program: 02.03.01 Mathematics and Computer Science

Laboratory Report No. 6
on the course "Graph Theory"
"Implementation of dictionaries based on a red-black tree and a hash table"

Student,
group 5130201/40001 _____ **Ovi Md Shamin Yasir**

Instructor,
_____ **Vostrov Alexey Vladimirovich**

"____"_____ **2026**

Saint Petersburg, 2026

Contents

Introduction

This report describes the implementation of Laboratory Work No. 6 for the course “Graph Theory”. The task was to implement a program that builds dictionaries based on a red-black tree and a hash table and performs various operations on them.

The first dictionary is built on a red-black tree. Functions for adding, deleting, and searching for words were implemented without using built-in data structures (map, set, etc.). Functions for clearing the dictionary and loading/appending from a text file are also provided.

The second dictionary is based on a hash table with the chaining method for collision resolution. A polynomial (non-trivial) hash function was chosen. It supports adding, deleting, searching, clearing, and loading/appending the dictionary from a text file.

The source text used is a poem by A. S. Pushkin — “I loved you” (“Я вас любил”), encoded in UTF-8 format.

1 Problem Statement

1. Dictionary based on a red-black tree

1. For an arbitrary Russian-language text, build a dictionary based on a red-black tree.
2. Implement functions for adding, deleting, and searching for words.
3. Built-in data structures must not be used.
4. Add a function for clearing the dictionary completely.
5. Add a function for loading and appending the dictionary from a text file.

2. Dictionary based on a hash table

1. For the same Russian-language text, implement a hash table supporting operations of adding, deleting, and searching.
2. Build a dictionary based on it.
3. Choose an arbitrary non-trivial hash function (describe its quality and collision resistance in the report).
4. Add a function for clearing the dictionary completely.
5. Add a function for loading and appending the dictionary from a text file.

2 Mathematical Description

2.1 Red-Black Tree

2.1.1 Definition of a Red-Black Tree

A **red-black tree** is a binary search tree whose balance is achieved by maintaining a coloring of nodes in two colors, subject to the following rules:

1. Each node is colored either red or black.
2. The root is always black.
3. Leaves are NIL nodes ("virtual" nodes), which are always black.
4. If a node is red, both its children must be black (no two consecutive red nodes).
5. All paths from the root to any leaf contain the same number of black nodes (black-height property).

To implement this type of balanced tree, each node stores an additional 1 bit of information (color).

2.1.2 Properties of Red-Black Trees

- The root and empty nodes (NIL) are always black.
- A red node cannot have red children.
- The maximum "black" lengths of all paths from the root to the leaves are equal.

Red-black search trees are sufficiently efficient: the number of steps in a search does not exceed $2 \log_2 n$.

2.1.3 Black Height

The **black height** of a node is the number of black nodes on the path from the given node to any leaf, not counting the node itself.

The fundamental property of a red-black tree: for any node, all paths from it to the leaves contain the same number of black nodes. This property guarantees the tree's balance.

Red nodes are not counted when calculating black height — they are attached to their black parents.

2.1.4 Rotations

Left and right rotations are used to restore the properties of a red-black tree.

Left rotation: raises the right child of node x to its position. A left rotation around node x moves its right child y to x's position, and x becomes the left child of y.

Right rotation: raises the left child of node y to its position. A right rotation is the mirror operation.

2.1.5 Node Insertion

When inserting, a new node is always colored **red**. After insertion, a property restoration procedure is called.

Three cases are possible (and three mirror cases):

1. **Case 1:** Uncle is red. Recoloring is performed — the parent and uncle become black, the grandparent becomes red. The check continues from the grandparent.
2. **Case 2:** Uncle is black, and the new node is "internal", i.e., the right child of the parent when the parent is the left child of the grandparent. A rotation of the parent is performed, after which we proceed to case 3.
3. **Case 3:** Uncle is black, and the new node is "external", i.e., the left child of the parent when the parent is the left child of the grandparent. A rotation of the grandparent and recoloring are performed.

2.1.6 Deletion from a Red-Black Tree

When deleting a node from a red-black tree, two cases are possible:

1. If the node being deleted is **red**, deletion is performed without violating properties.
2. If the node being deleted is **black**, the black height on paths through this node decreases, and restoration via delete fixup is required.

If the node being deleted has two children, its in-order successor (minimum in the right subtree) is found, its value is copied, and the successor is deleted.

Delete fixup analyzes the **sibling** of the deleted node and, depending on its color and the colors of its children, performs recoloring and rotations to restore balance. There are 4 cases (and 4 mirror cases):

1. Sibling is red — recolor and rotate.
2. Sibling is black, both children are black — recolor sibling, move up.
3. Sibling is black, far child is black — rotate sibling, reduce to case 4.
4. Sibling is black, far child is red — final rotation and recolor.

2.1.7 Complexity of Operations

- **Insertion:** $O(\log_2 n)$.
- **Search:** $O(\log_2 n)$.
- **Deletion:** $O(\log_2 n)$.
- **Clearing:** $O(n)$.

The number of steps in a search does not exceed $2 \log_2 n$.

Red-Black Tree Example

The following page (Fig. 1) shows the structure of the red-black tree after loading A. S. Pushkin's poem and adding the word "ovi" (37 words). Black nodes are shown as black ellipses, red nodes as red ellipses, and NIL leaves as black rectangles.

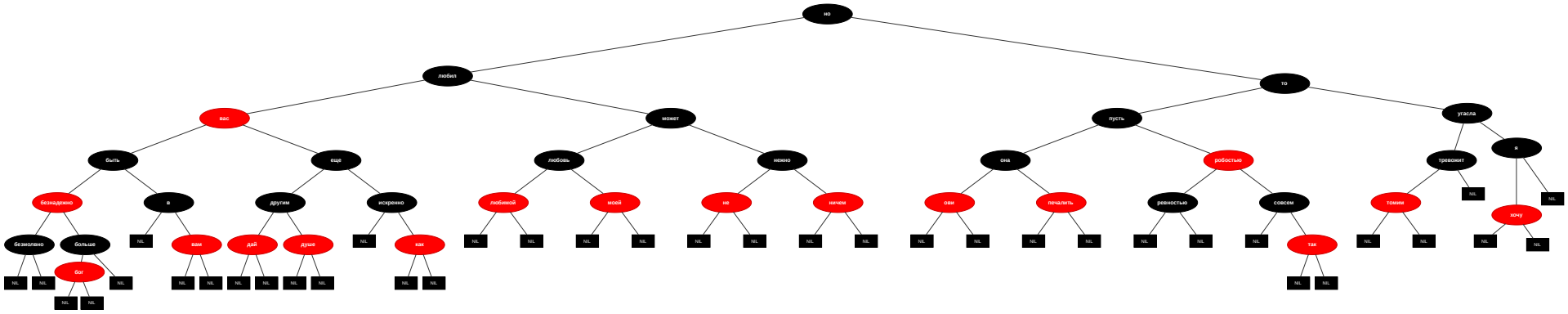

Рис. 1. Red-black tree (37 words)

∞

2.2 Hash Table

2.2.1 Definition of a Hash Table

A **hash table** is a data structure that implements an associative array interface, allowing storage of (key, value) pairs and performing search, insertion, and deletion operations in time close to $O(1)$.

A hash table consists of:

- An array of cells (buckets) of fixed size;
- A hash function that maps a key to an array index;
- A collision resolution method for cases when different keys receive the same hash.

2.2.2 Hash Function

A **hash function** is a function that maps the set of keys to the set of hash table indices:

$$h : K \to \{0, 1, \ldots, M-1\}$$

where K is the set of all possible keys, M is the hash table size.

In this work, a polynomial hash function is used:

$$H(s) = \left(\sum_{i=0}^{n-1} s_i \cdot 31^{n-1-i} \right) \bmod M$$

where:

- s is the input word (string);
- $n = |s|$ is the string length;
- s_i is the numeric code of the i-th byte of the word;
- 31 is a prime number (base);
- $M = 127$ is the hash table size (a prime number).

Why base 31 was chosen:

1. 31 is a prime number, which ensures better distribution of hash values and fewer collisions;
2. $31 = 2^5 - 1$, so multiplication by 31 can be optimized by the compiler to $(\text{hash} \ll 5) - \text{hash}$ (faster than general multiplication);
3. Small enough to avoid integer overflow, yet large enough to spread values across the table;
4. Odd primes work better than even numbers because multiplying by an even number shifts bits left and loses information.

Why table size is 127: it is a prime number, and $\gcd(31, 127) = 1$, which guarantees uniform distribution across buckets without clustering.

Hash function properties

1. For the same word, the function always returns the same index.
2. Hashes are distributed uniformly across the table.
3. Computation is fast, since complexity is $O(n)$, where n is the word length.

Hash function quality: The polynomial hash function uses *all* characters of the word, not just the first character (which would be a trivial approach). For 36 unique words from the test text, only 4 collisions occurred with 127 buckets (load factor 0.28), which indicates good function quality.

2.2.3 Collision Resolution — Chaining Method

Each table cell contains a pointer to a linked list of elements with the same hash value. On collision, a new element is added to the head of the list.

2.2.4 Complexity of Operations

- **Insertion:** $O(1)$ average, $O(n)$ worst case when all keys collide into one bucket.
- **Search:** $O(1)$ average, $O(n)$ worst case.
- **Deletion:** $O(1)$ average, $O(n)$ worst case.
- **Clearing:** $O(M)$, where M is the table size.

Таблица 1. Hash table example (fragment)

Key	Count	Hash
как	1	2
любовь	1	8
я	4	18
вас	5	33
так	2	69
не	3	115
любил	3	108
быть	2	126

Hash Table Example

Figs. 2 and 3 show the contents of the hash table after loading the poem. Collisions are marked with `[collision]` (buckets 17, 44, 86, 124).

```
Choice:12
 12
=== Hash Table Dictionary ===
  [2] как(2)
  [3] хочу(2)
  [8] любовь(2)
  [10] пусть(2)
  [11] может(2)
  [12] дай(2)
  [13] то(4)
  [17] ревностью(2) -> моей(2)  [collision]
  [22] в(2)
  [31] любимой(2)
  [33] вас(10)
  [41] другим(2)
  [44] безмолвно(2) -> еще(2)  [collision]
  [52] ови(1)
  [60] совсем(2)
  [61] вам(2)
  [62] печалить(2)
  [67] бог(2)
  [69] так(4)
  [70] нежно(2)
  [74] она(2)
  [77] тревожит(2)
  [86] больше(2) -> душе(2)  [collision]
  [99] ничем(2)
  [103] искренно(2)
  [104] безнадежно(2)
  [107] угасла(2)
  [108] любил(6)
  [115] не(6)
  [122] робостью(2)
  [124] томим(2) -> но(2)  [collision]
```

Рис. 2. Hash table structure (beginning)

```
  [126] быть(4)
Total unique words: 36
```

Рис. 3. Hash table structure (end)

Fig. 4 shows the statistics: size 127, 32 buckets used, 36 words, load factor 0.28, 4 total collisions, maximum chain length 2.

```
Choice:13
 13
=== Hash Table Statistics ===
Table size: 127
Used buckets: 32/127
Total unique words: 36
Load factor: 0.283465
Total collisions: 4
Max chain length: 2
Hash function: polynomial (base=31, mod=127)
```

Рис. 4. Hash table statistics

3 Implementation Details

The program is split into files: `rbtree.h/cpp` (red-black tree), `hashtable.h/cpp` (hash table), `textutils.h/cpp` (text processing), and `main.cpp` (menu). All data structures are implemented manually, without using built-in containers (map, set, unordered_map).

3.1 Data Structures

Two main data structures are implemented in the program.

3.1.1 Red-Black Tree, class `RBTree`

Description: The red-black tree is implemented using `RBNode` structures. Each node contains a word, an occurrence counter, a color (RED/BLACK), and pointers to the left child, right child, and parent. A sentinel NIL node (always black) is used.

```
enum Color { RED, BLACK };

struct RBNode {
    string word;
    int count;
    Color color;
    RBNode* left;
    RBNode* right;
    RBNode* parent;
};
```

Листинг 1. RBNode structure

The tree stores a pointer to the root and the sentinel:

```
RBNode* root;
RBNode* NIL;
```

3.1.2 Hash Table, class `HashTable`

Description: The hash table is stored as a fixed-size array, each element of which is a pointer to a linked list. This implements the chaining method for collision resolution.

```
const int TABLE_SIZE = 127;

struct HashNode {
    string word;
    int count;
    HashNode* next;
};

HashNode* table[TABLE_SIZE];
```

3.1.3 Utilities

Description: Auxiliary functions for processing Russian text in UTF-8 encoding. Used by both data structures.

```
string toLowerRussian(const string& text);
bool isRussianLetter(const string& text, size_t i);
bool isAllRussian(const string& text);
void processText(const string& text, RBTree& tree, HashTable& ht);
```

3.2 Red-Black Tree

This module implements functions for the dictionary based on a red-black tree.

3.2.1 rotateLeft and rotateRight

Input: red-black tree and node `x` around which the rotation is performed.
Output: tree with modified structure, while key order is preserved.
Description: A left rotation around node x moves its right child y to x's position. A right rotation is the mirror operation.

```
void RBTree::rotateLeft(RBNode* x) {
    RBNode* y = x->right;
    x->right = y->left;
    if (y->left != NIL) y->left->parent = x;
    y->parent = x->parent;
    if (x->parent == nullptr) root = y;
    else if (x == x->parent->left) x->parent->left = y;
    else x->parent->right = y;
    y->left = x;
    x->parent = y;
}

void RBTree::rotateRight(RBNode* x) {
    RBNode* y = x->left;
    x->left = y->right;
    if (y->right != NIL) y->right->parent = x;
    y->parent = x->parent;
    if (x->parent == nullptr) root = y;
    else if (x == x->parent->right) x->parent->right = y;
    else x->parent->left = y;
    y->right = x;
    x->parent = y;
}
```

Листинг 2. rotateLeft and rotateRight functions

3.2.2 insertFixup

Input: red-black tree and the inserted node `z` (red).
Output: tree with restored properties (no two consecutive reds, equal black height).
Description: Restores RB-tree properties after insertion. Runs in a loop while there are two consecutive red nodes. Depending on the uncle's color, performs recoloring or rotations. Handles 3 cases and their mirror reflections.

```
void RBTree::insertFixup(RBNode* z) {
    while (z->parent != nullptr && z->parent->color == RED) {
        if (z->parent == z->parent->parent->left) {
            RBNode* uncle = z->parent->parent->right;
            if (uncle->color == RED) {
                z->parent->color = BLACK;
                uncle->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            } else {
                if (z == z->parent->right) {
                    z = z->parent;
                    rotateLeft(z);
                }
```

```
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                rotateRight(z->parent->parent);
            }
        } else { /* Mirror cases */ }
    }
    root->color = BLACK;
}
```

Листинг 3. insertFixup function

3.2.3 insert

Input: red-black tree and word `word`.
Output: tree is augmented with a new node while maintaining balance.
Description: If the word already exists — increments the counter. Otherwise, creates a new red node, performs standard BST insertion, then calls `insertFixup`.

```
void RBTree::insert(const string& word) {
    RBNode* existing = searchNode(word);
    if (existing != NIL) { existing->count++; return; }

    RBNode* z = new RBNode();
    z->word = word; z->count = 1; z->color = RED;
    z->left = NIL; z->right = NIL; z->parent = nullptr;

    RBNode* y = nullptr;
    RBNode* x = root;
    while (x != NIL) {
        y = x;
        if (word < x->word) x = x->left;
        else x = x->right;
    }
    z->parent = y;
    if (y == nullptr) root = z;
    else if (word < y->word) y->left = z;
    else y->right = z;

    if (z->parent == nullptr) { z->color = BLACK; return; }
    if (z->parent->parent == nullptr) return;
    insertFixup(z);
}
```

Листинг 4. insert function

3.2.4 searchNode

Input: red-black tree and word `word`.
Output: pointer to the found node, or NIL if the word is not present.
Description: Standard binary tree search: move left or right depending on comparison.

```
RBNode* RBTree::searchNode(const string& word) {
    RBNode* current = root;
    while (current != NIL) {
        if (word == current->word) return current;
        else if (word < current->word) current = current->left;
        else current = current->right;
    }
    return NIL;
```

```
}
```

Листинг 5. searchNode function

3.2.5 deleteFixup

Input: red-black tree and node `x` (replacement of the deleted node).

Output: tree with restored black height.

Description: Restores properties after deletion of a black node. Runs in a loop, analyzing the sibling and its children. Performs recoloring and rotations to maintain balance. 4 cases and 4 mirror cases.

```
void RBTree::deleteFixup(RBNode* x) {
    while (x != root && x->color == BLACK) {
        if (x == x->parent->left) {
            RBNode* w = x->parent->right;
            if (w->color == RED) { /* Case 1 */
                w->color = BLACK; x->parent->color = RED;
                rotateLeft(x->parent); w = x->parent->right;
            }
            if (w->left->color == BLACK && w->right->color == BLACK) {
                w->color = RED; x = x->parent; /* Case 2 */
            } else {
                if (w->right->color == BLACK) { /* Case 3 */
                    w->left->color = BLACK; w->color = RED;
                    rotateRight(w); w = x->parent->right;
                }
                /* Case 4 */
                w->color = x->parent->color;
                x->parent->color = BLACK;
                w->right->color = BLACK;
                rotateLeft(x->parent); x = root;
            }
        } else { /* Mirror cases */ }
    }
    x->color = BLACK;
}
```

Листинг 6. deleteFixup function (left branch)

3.2.6 remove

Input: red-black tree and word `word`.

Output: `true` on successful deletion, `false` if not found; tree maintains RB-tree properties.

Description: Handles three cases: node without left child, without right child, and with two children (via in-order successor). If a black node is deleted — `deleteFixup` is called.

```
bool RBTree::remove(const string& word) {
    RBNode* z = searchNode(word);
    if (z == NIL) return false;
    RBNode* y = z; RBNode* x;
    Color yOriginalColor = y->color;
    if (z->left == NIL) {
        x = z->right; transplant(z, z->right);
    } else if (z->right == NIL) {
        x = z->left; transplant(z, z->left);
    } else {
```

```
        y = treeMinimum(z->right);
        yOriginalColor = y->color; x = y->right;
        if (y->parent == z) { x->parent = y; }
        else {
            transplant(y, y->right);
            y->right = z->right; y->right->parent = y;
        }
        transplant(z, y);
        y->left = z->left; y->left->parent = y;
        y->color = z->color;
    }
    delete z;
    if (yOriginalColor == BLACK) deleteFixup(x);
    return true;
}
```

Листинг 7. remove function

3.2.7 printTreeHelper

Input: current node `node`, prefix string `prefix`, flags `isLeft` and `isRoot`.

Output: a fragment of the tree structure with node colors and NIL leaves is printed to the screen.

Description: Recursively prints the tree sideways: right subtree on top, left subtree on bottom. Box-drawing characters are used to display connections between nodes.

```
void RBTree::printTreeHelper(RBNode* node,
        const string& prefix, bool isLeft, bool isRoot) {
    if (node == NIL) {
        if (isRoot) cout << "[B] NIL" << endl;
        else cout << prefix << (isLeft ? "L-- " : "|-- ")
                  << "[B] NIL" << endl;
        return;
    }
    string rightPfx = isRoot ? "  "
        : (prefix + (isLeft ? "|   " : "    "));
    printTreeHelper(node->right, rightPfx, false, false);
    string label = (node->color == RED ? "[R] " : "[B] ");
    if (isRoot) cout << label << node->word
                     << "(" << node->count << ")" << endl;
    else cout << prefix << (isLeft ? "L-- " : "|-- ")
              << label << node->word
              << "(" << node->count << ")" << endl;
    string leftPfx = isRoot ? "  "
        : (prefix + (isLeft ? "    " : "|   "));
    printTreeHelper(node->left, leftPfx, true, false);
}
```

Листинг 8. Tree visualization

3.3 Hash Table

This module implements functions for the dictionary based on a hash table.

3.3.1 hashFunction

Input: hash table and key — word `word`.

Output: integer — bucket index from 0 to `TABLE_SIZE-1`.

Description: Polynomial hash function with base 31. Uses all characters of the word.

```
unsigned int HashTable::hashFunction(const string& word) {
    unsigned int hash = 0;
    for (size_t i = 0; i < word.length(); i++) {
        hash = hash * 31 + (unsigned char)word[i];
    }
    return hash % TABLE_SIZE;
}
```

Листинг 9. hashFunction

3.3.2 insert

Input: hash table and word `word` to add.

Output: dictionary is augmented with a new word or the counter of an existing one is incremented.

Description: Computes the hash, checks the chain for the word. If found — increments the counter. If not found — creates a new node at the head of the chain.

```
void HashTable::insert(const string& word) {
    unsigned int index = hashFunction(word);
    HashNode* current = table[index];
    while (current != nullptr) {
        if (current->word == word) { current->count++; return; }
        current = current->next;
    }
    HashNode* newNode = new HashNode();
    newNode->word = word; newNode->count = 1;
    newNode->next = table[index];
    table[index] = newNode;
}
```

Листинг 10. Hash table insertion

3.3.3 search

Input: hash table and word `word` to search for.

Output: `true` if the word is in the dictionary; `false` otherwise.

Description: Computes the hash and performs a linear search in the corresponding chain.

```
bool HashTable::search(const string& word) {
    unsigned int index = hashFunction(word);
    HashNode* current = table[index];
    while (current != nullptr) {
        if (current->word == word) return true;
        current = current->next;
    }
    return false;
}
```

Листинг 11. Hash table search

3.3.4 remove

Input: hash table and word `word` to delete.

Output: the specified word is removed from the dictionary if present.

Description: Finds the word in the chain and removes the node with pointer adjustment.

```
bool HashTable::remove(const string& word) {
    unsigned int index = hashFunction(word);
    HashNode* current = table[index];
    HashNode* prev = nullptr;
    while (current != nullptr) {
        if (current->word == word) {
            if (prev == nullptr) table[index] = current->next;
            else prev->next = current->next;
            delete current; return true;
        }
        prev = current; current = current->next;
    }
    return false;
}
```

Листинг 12. Hash table deletion

3.3.5 printStats

Input: hash table.

Output: table size, number of used buckets, load factor, number of collisions, and maximum chain length are printed to the screen.

Description: Prints hash table statistics: traverses all buckets, counts used ones, collisions, and maximum chain length.

```
void HashTable::printStats() {
    int usedBuckets = 0, totalWords = 0;
    int maxChain = 0, collisions = 0;
    for (int i = 0; i < TABLE_SIZE; i++) {
        int chainLen = 0;
        HashNode* current = table[i];
        while (current != nullptr) { chainLen++; current = current->next; }
        if (chainLen > 0) usedBuckets++;
        if (chainLen > 1) collisions += (chainLen - 1);
        if (chainLen > maxChain) maxChain = chainLen;
        totalWords += chainLen;
    }
    cout << "Table size: " << TABLE_SIZE << endl;
    cout << "Used buckets: " << usedBuckets << "/" << TABLE_SIZE << endl;
    cout << "Load factor: " << (double)totalWords / TABLE_SIZE << endl;
    cout << "Total collisions: " << collisions << endl;
}
```

Листинг 13. printStats function

3.4 Utilities

This module implements auxiliary functions for processing Russian text in UTF-8 encoding.

3.4.1 isRussianLetter and processText

Input: a UTF-8 string with Russian text, the position of the character being checked, and references to both dictionaries.

Output: Russian words are extracted from the text, converted to lowercase, and added to the red-black tree and hash table.

Description: The function `isRussianLetter` checks whether a two-byte UTF-8 sequence is a Russian letter. The function `processText` extracts words from text and inserts them into both dictionaries.

```
bool isRussianLetter(const string& text, size_t i) {
    if (i >= text.size()) return false;
    unsigned char c = text[i];
    if (c == 0xD0 && i + 1 < text.size()) {
        unsigned char next = text[i + 1];
        if ((next >= 0x90 && next <= 0xBF) || next == 0x81) return true;
    }
    if (c == 0xD1 && i + 1 < text.size()) {
        unsigned char next = text[i + 1];
        if ((next >= 0x80 && next <= 0x8F) || next == 0x91) return true;
    }
    return false;
}

void processText(const string& text, RBTree& tree, HashTable& ht) {
    string word;
    size_t i = 0;
    while (i < text.size()) {
        if (isRussianLetter(text, i)) {
            word += text[i]; word += text[i + 1]; i += 2;
        } else {
            if (!word.empty()) {
                string lower = toLowerRussian(word);
                tree.insert(lower); ht.insert(lower);
                word.clear();
            }
            i++;
        }
    }
    if (!word.empty()) {
        string lower = toLowerRussian(word);
        tree.insert(lower); ht.insert(lower);
    }
}
```

Листинг 14. Russian text processing

4 Program Results

4.1 Main Menu

Fig. 5 shows the main menu of the program with 14 available operations for both dictionaries.

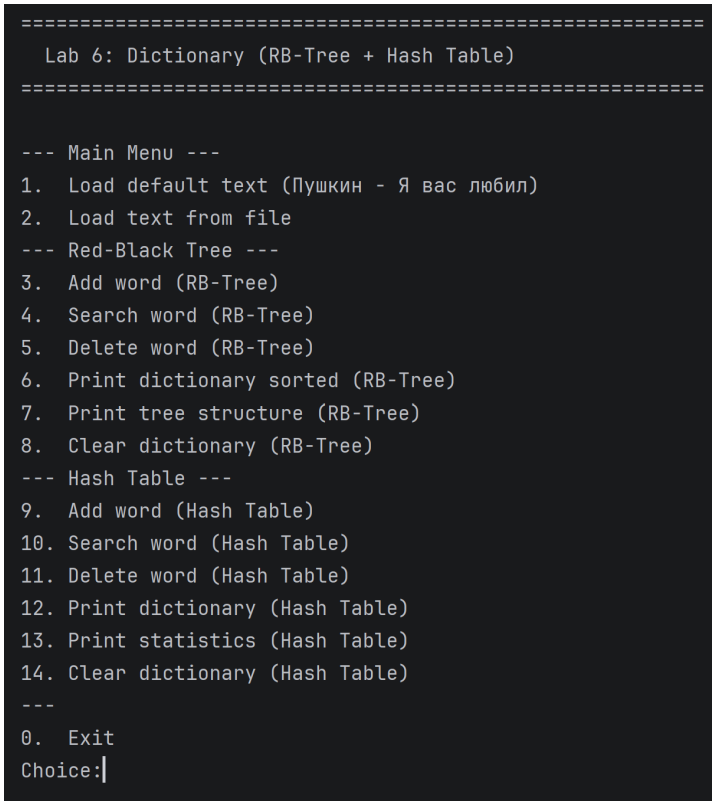

Рис. 5. Program main menu

4.2 Loading Text (option 1)

Fig. 6 shows loading the poem by A. S. Pushkin “I loved you” into both dictionaries simultaneously.

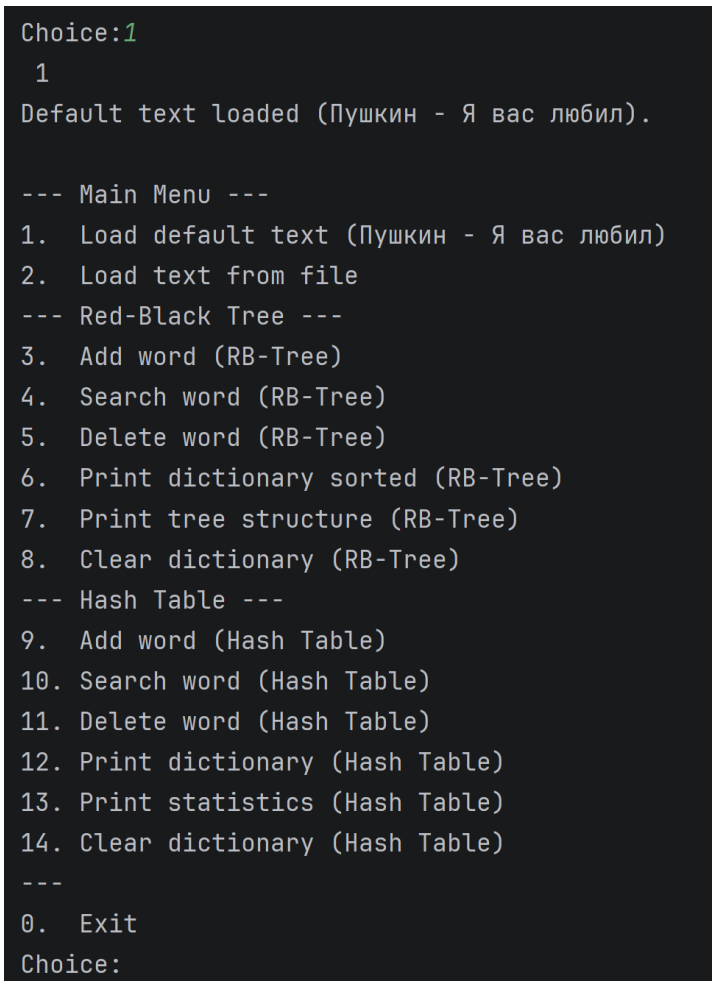

Рис. 6. Loading default text

4.3 Adding a Word to the RB-Tree (option 3)

Figs. 7–9 show adding the word “Ovi” to the RB-tree and the tree visualization after insertion. The word is converted to lowercase and inserted as a red node.

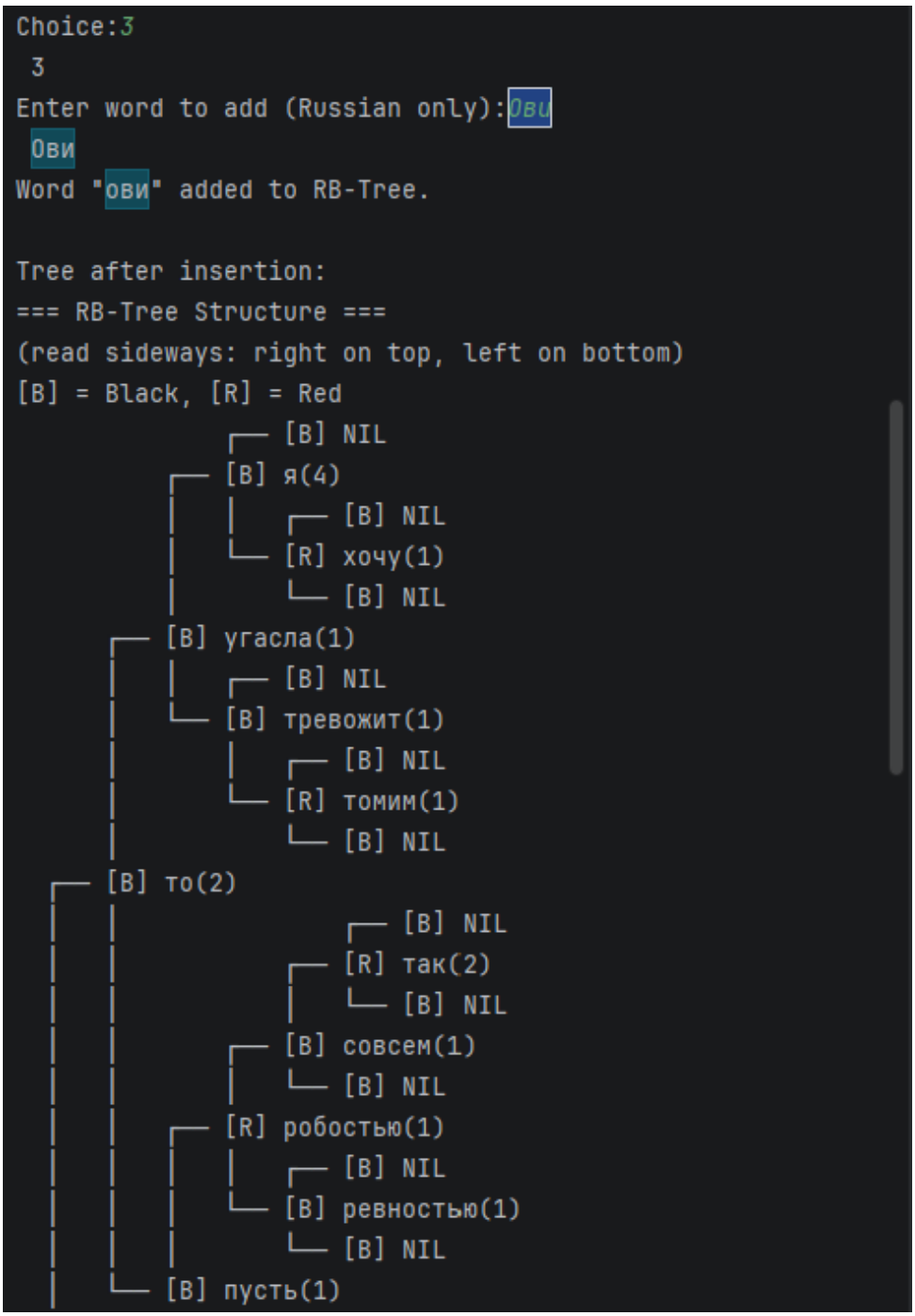

Рис. 7. Adding a word to the RB-tree (top part of the tree)

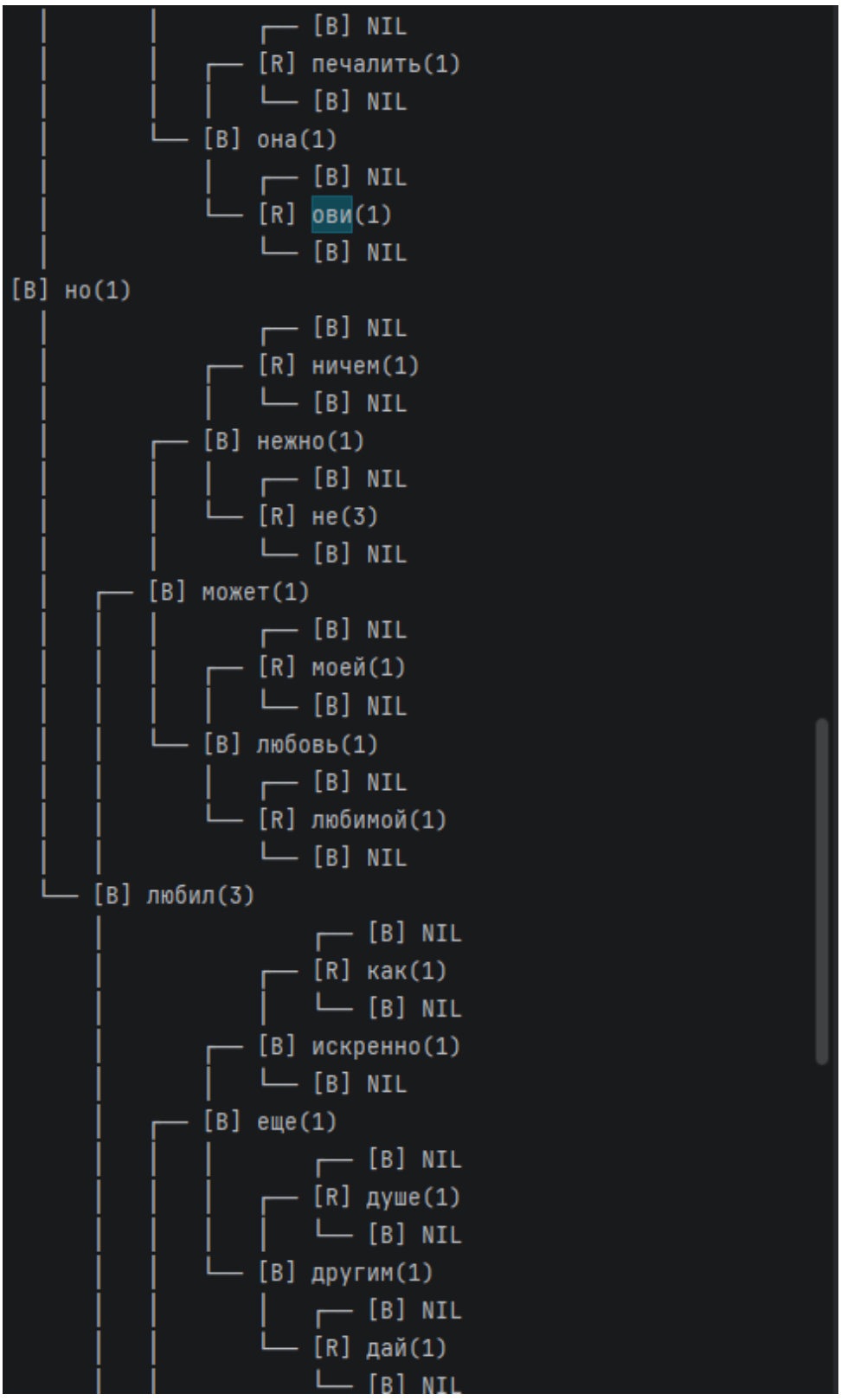

Рис. 8. Tree after insertion (middle part)

Рис. 9. Tree after insertion (bottom part)

4.4 Searching for a Word in the RB-Tree (option 4)

Fig. 10 shows searching for the word “я” in the RB-tree. The word was found with an occurrence count of 4.

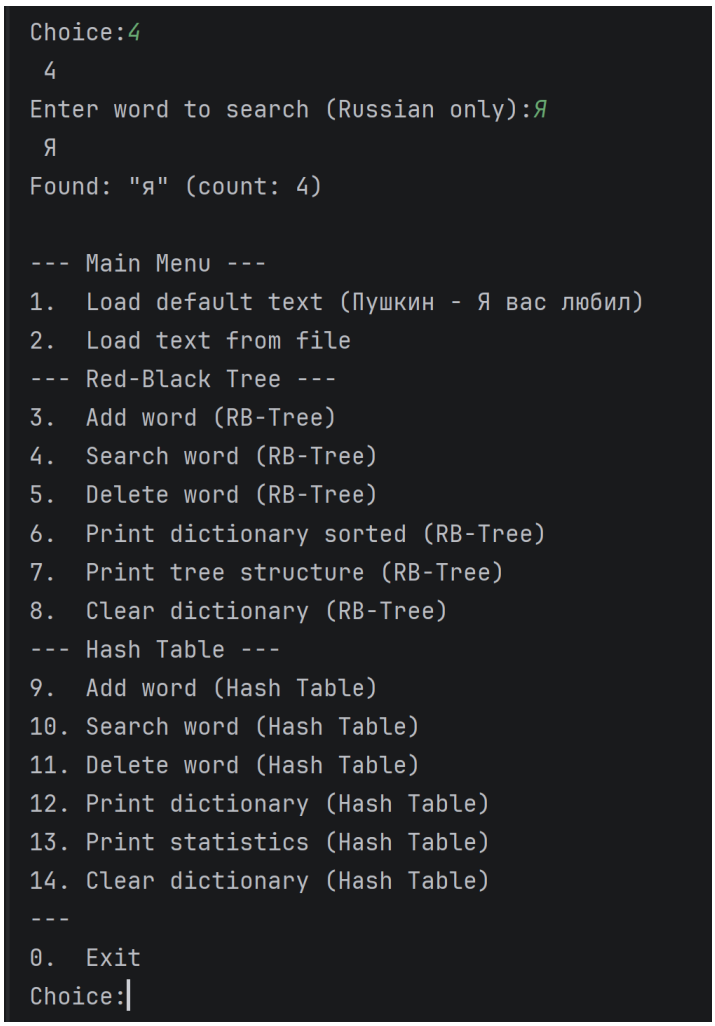

Рис. 10. Searching for a word in the RB-tree

4.5 Deleting a Word from the RB-Tree (option 5)

Figs. 11 and 12 show the tree state before and after deletion of a node. After deletion, the tree is rebalanced and retains all red-black tree properties.

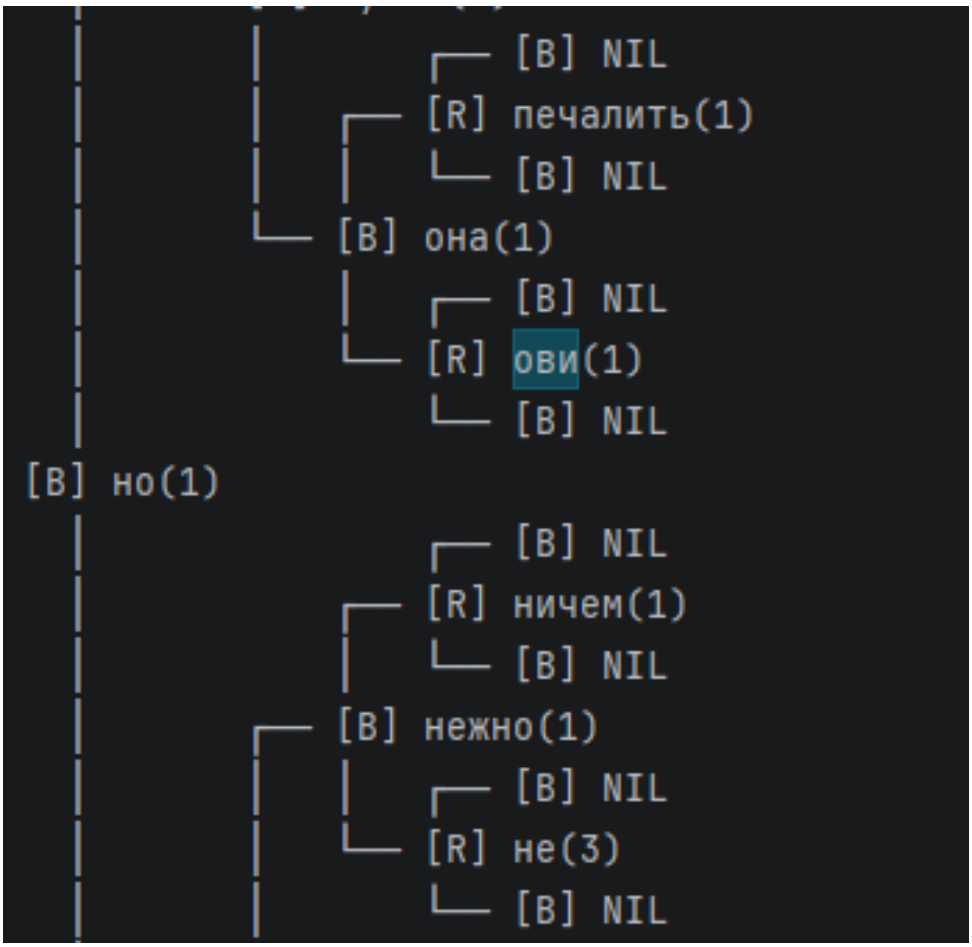

Рис. 11. Tree fragment BEFORE deletion

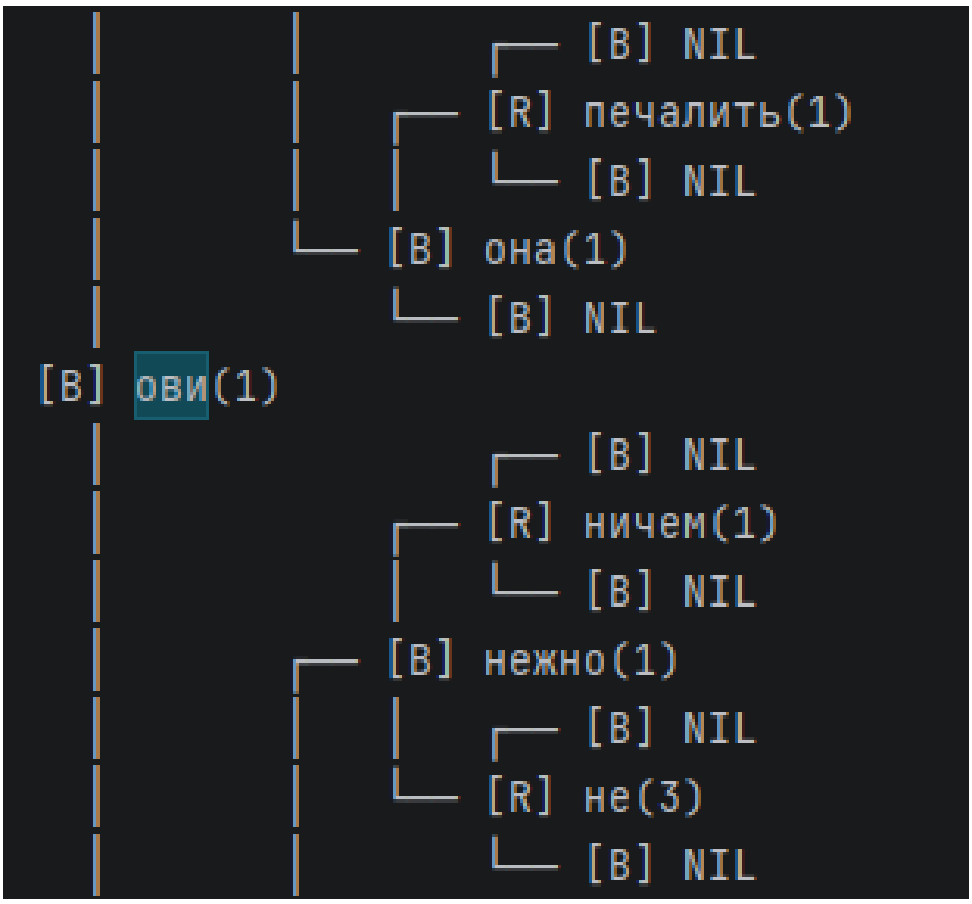

Рис. 12. Tree fragment AFTER deletion

4.6 Printing the RB-Tree Dictionary (option 6)

Figs. 13 and 14 show the sorted dictionary (in-order tree traversal). A total of 36 unique words.

```
Choice:6
 6
=== RB-Tree Dictionary (sorted) ===
  безмолвно : 1
  безнадежно : 1
  бог : 1
  больше : 1
  быть : 2
  в : 1
  вам : 1
  вас : 5
  дай : 1
  другим : 1
  душе : 1
  еще : 1
  искренно : 1
  как : 1
  любил : 3
  любимой : 1
  любовь : 1
  моей : 1
  может : 1
  не : 3
  нежно : 1
  ничем : 1
  но : 1
  она : 1
  печалить : 1
  пусть : 1
  ревностью : 1
```

Рис. 13. RB-tree dictionary, sorted (beginning)

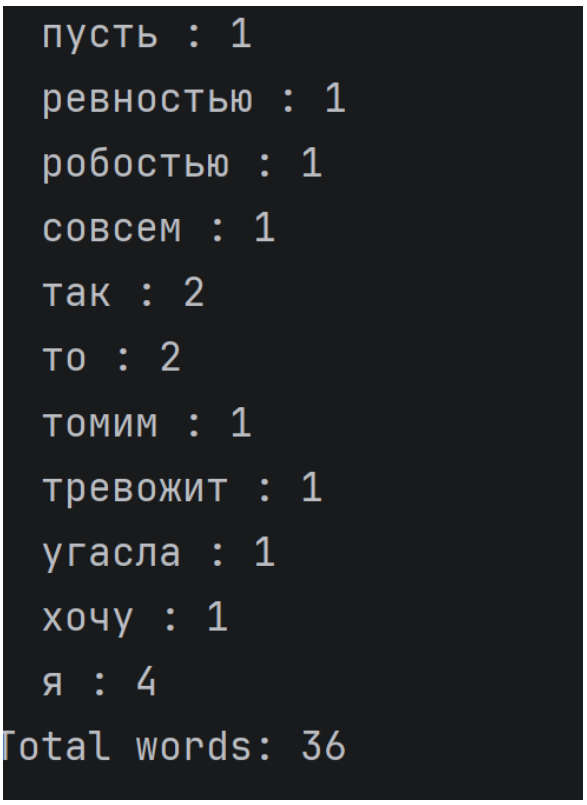

Рис. 14. RB-tree dictionary, sorted (end)

4.7 RB-Tree Structure (option 7)

Figs. 15–17 show the tree structure visualization. The tree is printed sideways: right subtree on top, left subtree on the bottom. Nodes are labeled [R] (red) and [B] (black); NIL leaves are displayed as black nodes.

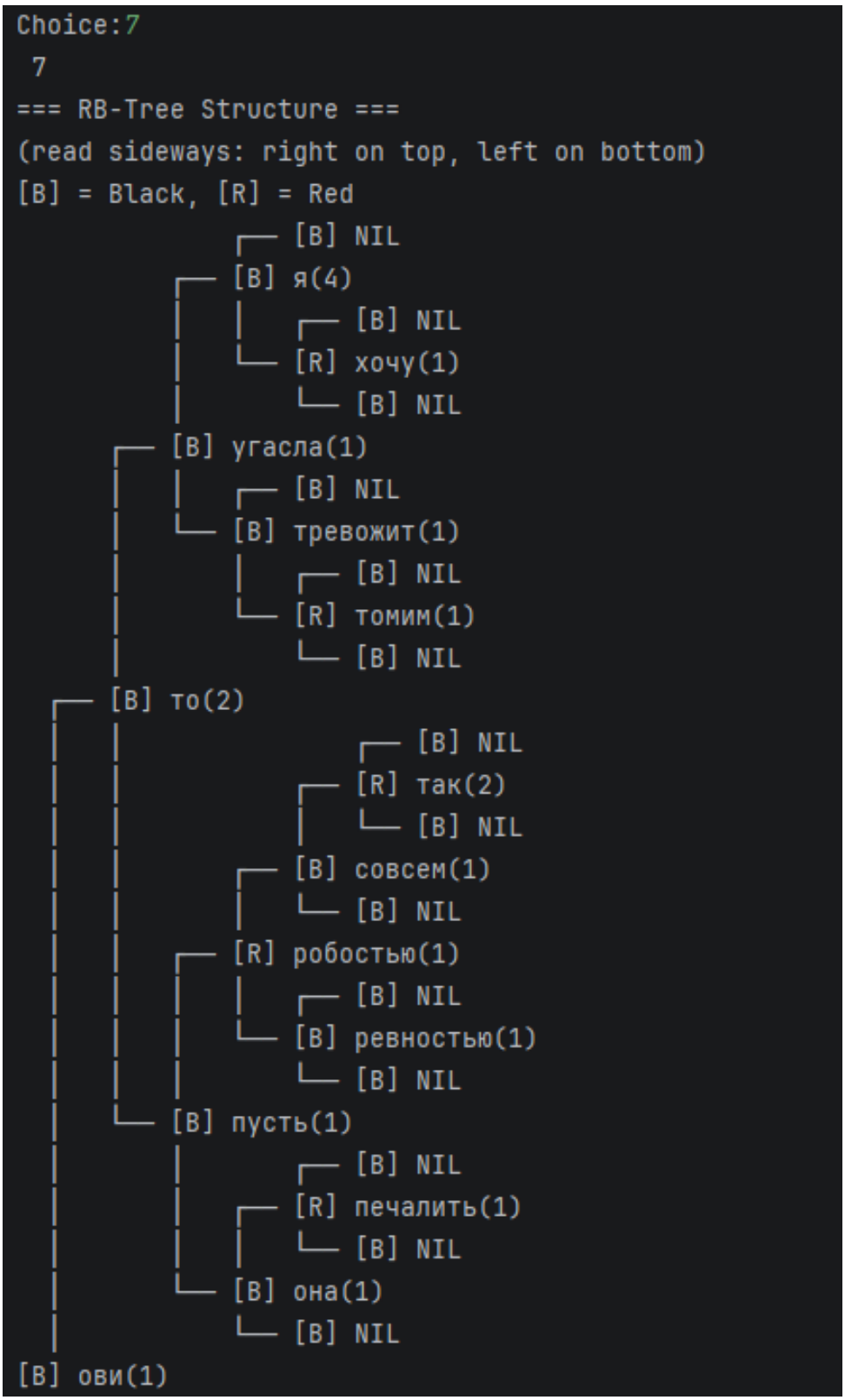

Рис. 15. RB-tree structure (top part)

Рис. 16. RB-tree structure (middle part)

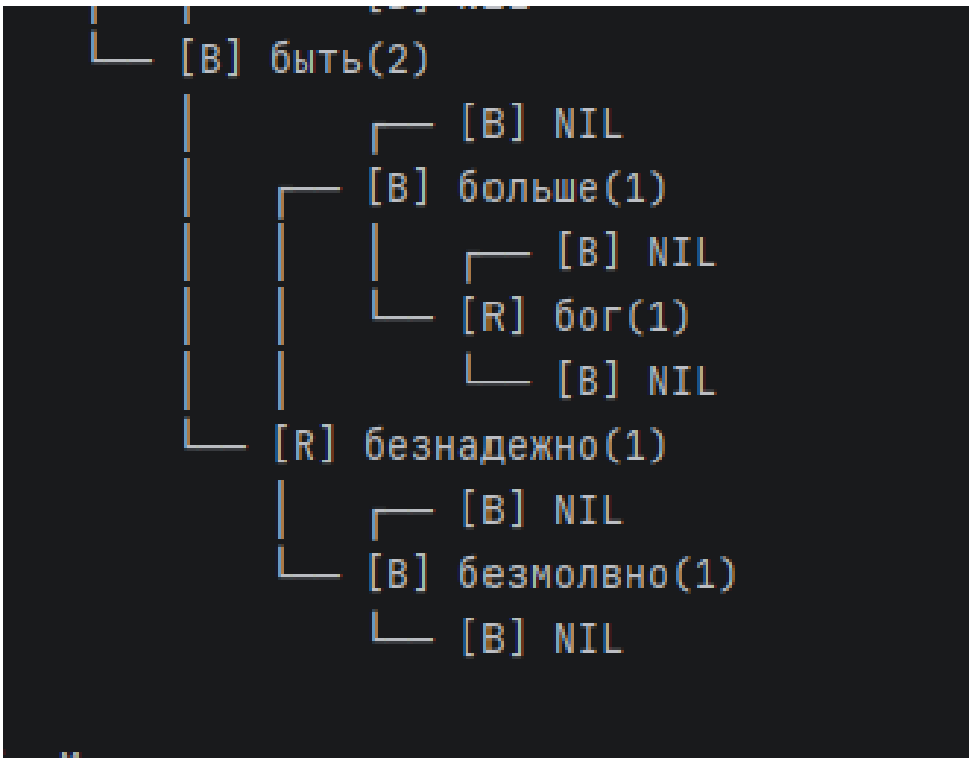

Рис. 17. RB-tree structure (bottom part)

4.8 Clearing the RB-Tree (option 8)

Fig. 18 shows clearing the RB-tree dictionary.

```
Choice:8
 8
RB-Tree dictionary cleared.
```

Рис. 18. Clearing the RB-tree dictionary

4.9 Adding a Word to the Hash Table (option 9)

Fig. 19 shows adding the word “Ovi” to the hash table. The word is converted to lowercase.

```
Choice:9
 9
Enter word to add (Russian only):Ови
 Ови
Word "ови" added to Hash Table.
```

Рис. 19. Adding a word to the hash table

4.10 Searching for a Word in the Hash Table (option 10)

Fig. 20 shows searching for the word “я” in the hash table. The occurrence count and bucket number (bucket 18) are displayed.

```
Choice:10
 10
Enter word to search (Russian only):Я
 Я
Found: "я" (count: 8, bucket: 18)
```

Рис. 20. Searching for a word in the hash table

4.11 Deleting a Word from the Hash Table (option 11)

Fig. 21 shows deleting the word “я” from the hash table (bucket 18).

```
Choice:11
 11
Enter word to delete (Russian only):Я
 Я
Deleted "я" from bucket 18
```

Рис. 21. Deleting a word from the hash table

4.12 Printing the Hash Table (option 12)

Figs. 22 and 23 show the contents of the hash table. Collisions are marked with the `[collision]` label (e.g., buckets 17, 44, 86, 124). A total of 36 unique words.

```
Choice:12
 12
=== Hash Table Dictionary ===
  [2] как(2)
  [3] хочу(2)
  [8] любовь(2)
  [10] пусть(2)
  [11] может(2)
  [12] дай(2)
  [13] то(4)
  [17] ревностью(2) -> моей(2)  [collision]
  [22] в(2)
  [31] любимой(2)
  [33] вас(10)
  [41] другим(2)
  [44] безмолвно(2) -> еще(2)  [collision]
  [52] ови(1)
  [60] совсем(2)
  [61] вам(2)
  [62] печалить(2)
  [67] бог(2)
  [69] так(4)
  [70] нежно(2)
  [74] она(2)
  [77] тревожит(2)
  [86] больше(2) -> душе(2)  [collision]
  [99] ничем(2)
  [103] искренно(2)
  [104] безнадежно(2)
  [107] угасла(2)
  [108] любил(6)
  [115] не(6)
  [122] робостью(2)
  [124] томим(2) -> но(2)  [collision]
```

Рис. 22. Hash table contents with collisions

```
  [126] быть(4)
Total unique words: 36
```

Рис. 23. Hash table contents (end)

4.13 Hash Table Statistics (option 13)

Fig. 24 shows hash table statistics: size 127, 32 buckets used, 36 words, load factor 0.28, 4 total collisions, maximum chain length 2.

```
Choice:13
 13
=== Hash Table Statistics ===
Table size: 127
Used buckets: 32/127
Total unique words: 36
Load factor: 0.283465
Total collisions: 4
Max chain length: 2
Hash function: polynomial (base=31, mod=127)
```

Рис. 24. Hash table statistics

4.14 Clearing the Hash Table (option 14)

Fig. 25 shows clearing the hash table.

```
Choice:14
 14
Hash Table dictionary cleared.
```

Рис. 25. Clearing the hash table

4.15 Exiting the Program (option 0)

Fig. 26 shows a clean program exit.

```
Choice:0
 0
Goodbye!
```

Рис. 26. Program exit

Conclusion

As a result of this work, two dictionaries for arbitrary Russian-language text were implemented:

1. A dictionary based on a red-black tree with functions for adding, deleting, searching, tree structure visualization, complete clearing, and loading from a text file.
2. A dictionary based on a hash table with chaining and a polynomial hash function (base 31, modulus 127). Functions for adding, deleting, searching, printing statistics, complete clearing, and loading from a file were implemented.

All data structures were implemented without using built-in STL containers.

The program correctly processes Russian text in UTF-8 encoding, converts words to lowercase, and validates user input.

Hash function characteristics:

- The polynomial hash function uses all characters of the word (non-trivial);
- For 36 unique words with a table size of 127, only 4 collisions occurred;
- Load factor: 0.28;
- Maximum chain length: 2.

RB-tree correctness:

- Verified by sequential deletion of 5 nodes, including the root;
- After each deletion, the tree maintains all 5 red-black tree properties;
- Tree visualization before and after deletion confirms correct rebalancing;
- NIL leaves are correctly displayed as black nodes.

Advantages:

- Both dictionaries operate on the same text and can load data from a file;
- RB-tree visualization allows clear demonstration of balancing;
- Input validation prevents adding non-Russian words and invalid commands;
- Base 31 and table size 127 were chosen such that $\gcd(31, 127) = 1$, which ensures uniform distribution across cells and minimal collisions.

Disadvantages:

- No graphical user interface;
- Fixed hash table size (127 cells).

Program scaling:

- Replacing the hash function to reduce collisions, e.g., using double hashing;
- Dynamic resizing of the hash table when the load factor exceeds a threshold;
- Saving the dictionary to a file and loading from a file in a structured format.

5 Detailed Walkthrough with Examples

This section explains how every part of the program works using real output from a test run with the words: а, б, в, г, д, е.

5.1 What is a Red-Black Tree?

A red-black tree is a **self-balancing binary search tree**. In a regular binary search tree (BST), if we insert sorted data (а, б, в, г, д, е), the tree becomes a straight line (like a linked list), and search takes $O(n)$. A red-black tree prevents this by coloring nodes red or black and rotating/recoloring after every insert and delete, so the tree stays approximately balanced and search always takes $O(\log_2 n)$.

The 5 rules that must always hold:

1. Every node is either red or black.
2. The root is always black.
3. NIL leaves (empty children) are black.
4. A red node cannot have a red child (no two reds in a row).
5. Every path from root to any NIL has the same number of black nodes (black height).

In our code (`rbtree.h`), each node stores:

```
struct RBNode {
    string word;      // the dictionary word
    int count;        // how many times it appeared
    Color color;      // RED or BLACK
    RBNode* left;     // left child
    RBNode* right;    // right child
    RBNode* parent;   // parent node
};
```

We also use a sentinel `NIL` node — a single black node that represents all empty positions. This simplifies the code because we never have to check for `nullptr`.

5.2 How Insertion Works — Step by Step

When we insert words one by one (а, б, в, г, д, е), here is what happens at each step:

Step 1: Insert "а". Tree is empty, so а becomes the root. Root must be black (rule 2).

```
[B] a(1)
```

Step 2: Insert "б". б > а, so it goes to the right. New nodes are always red.

```
   /-- [R] b(1)
[B] a(1)
```

No violation: red б has black parent а. OK.

Step 3: Insert "в". в > б, goes right of б. Now we have two reds in a row (б red, в red) — rule 4 is violated! The `insertFixup` function runs.

Since а's left child (uncle) is NIL (black), this is **Case 3**: left rotation around а, then recolor. Result: б becomes root (black), а and в become red children. But since both children of root are red and both their children are NIL (black), we can make а and в red:

```
   /-- [R] v(1)
[B] b(1)
   \-- [R] a(1)
```

Step 4–6: Insert г, д, е. Similar process. After each insert, `insertFixup` runs if needed. The final tree after all 6 words:

```
           /-- [R] e(1)
       /-- [B] d(1)
   /-- [R] g(1)          <-- right child of root
   |       \-- [B] v(1)
[B] b(1)                  <-- root (black)
   \-- [B] a(1)           <-- left child of root
```

The relevant code in `rbtree.cpp`:

```
void RBTree::insert(const string& word) {
    // 1. Check if word exists -- if yes, just increment count
    RBNode* existing = searchNode(word);
    if (existing != NIL) { existing->count++; return; }

    // 2. Create new RED node
    RBNode* z = new RBNode();
    z->word = word; z->count = 1; z->color = RED;
    z->left = NIL; z->right = NIL;

    // 3. Standard BST insert: find correct position
    RBNode* y = nullptr; RBNode* x = root;
    while (x != NIL) {
        y = x;
        if (word < x->word) x = x->left;
        else x = x->right;
    }
    z->parent = y;
    if (y == nullptr) root = z;         // tree was empty
    else if (word < y->word) y->left = z;
    else y->right = z;

    // 4. Fix any violations caused by red insertion
    if (z->parent == nullptr) { z->color = BLACK; return; }
    if (z->parent->parent == nullptr) return;
    insertFixup(z);  // <-- this does the rotations/recoloring
}
```

5.3 How Deletion Works — Deleting the Root “б”

This is the most complex operation. Let us trace through deleting б (the root) step by step.

Before deletion:

```
           /-- [R] e(1)
       /-- [B] d(1)
   /-- [R] g(1)
   |       \-- [B] v(1)
[B] b(1)               <-- deleting this node
   \-- [B] a(1)
```

Step 1: Find the in-order successor.

Since б has two children, we cannot simply remove it. We find the **in-order successor** — the smallest word that is larger than б. This is found by going right once (to г), then left as far as possible: г → в. The word в has no left child, so в is the successor.

In the code (`rbtree.cpp`):

```
y = treeMinimum(z->right);  // z is "b", z->right is "g"
// treeMinimum goes left until NIL:
// "g" -> left = "v" -> left = NIL -> return "v"
```

Step 2: Replace б with в.

The successor в is moved to б's position. в's old spot (left child of г) becomes NIL. в takes б's color (BLACK) and inherits б's children (а on left, г on right).

```
// From remove():
transplant(y, y->right);     // remove "v" from its position
y->right = z->right;         // "v"->right = "g"
transplant(z, y);             // put "v" where "b" was (root)
y->left = z->left;           // "v"->left = "a"
y->color = z->color;         // "v" gets BLACK ("b"'s color)
delete z;                     // free "b"'s memory
```

Step 3: Fix the tree (`deleteFixup`).

Since the removed node (в) was BLACK, the black height is broken. `deleteFixup` runs and performs rotations and recoloring to restore balance. After fixup:

```
       /-- [B] e(1)
   /-- [R] d(1)         <-- d became RED
   |       \-- [B] g(1)  <-- g became BLACK
[B] v(1)                  <-- new root
   \-- [B] a(1)
```

We can verify: black height = 2 on every path (в→д→е = B,R,B = 2 black; в→д→г = B,R,B = 2; в→а = B,B = 2). All 5 rules hold.

5.4 What is a Hash Table?

A hash table is an array where each position (called a **bucket**) can store words. A **hash function** converts a word into a number (the bucket index). This gives us $O(1)$ average-time lookup — much faster than searching through a list.

The problem: different words can produce the same index. This is called a **collision**. Our code resolves collisions using **chaining** — each bucket holds a linked list of all words that hash to that index.

In our code (`hashtable.h`):

```
const int TABLE_SIZE = 127;  // 127 buckets

struct HashNode {
    string word;       // the word
    int count;         // occurrence count
    HashNode* next;    // next node in chain (for collisions)
};

HashNode* table[TABLE_SIZE];  // array of 127 bucket pointers
```

5.5 Why Base 31 and Table Size 127?

Our hash function (`hashtable.cpp`):

```
unsigned int HashTable::hashFunction(const string& word) {
    unsigned int hash = 0;
    for (size_t i = 0; i < word.length(); i++) {
        hash = hash * 31 + (unsigned char)word[i];
    }
    return hash % TABLE_SIZE;  // TABLE_SIZE = 127
}
```

Why 31?

- 31 is a **prime number** — primes distribute hash values more evenly.
- $31 = 2^5 - 1$, so the compiler optimizes `hash * 31` to `(hash << 5) - hash` (fast bit shift instead of multiplication).
- It is the same constant used in Java's `String.hashCode()`.

Why 127?

- 127 is a **prime number**.
- $\gcd(31, 127) = 1$ (no common factors), which means values spread across all 127 buckets without clustering.

Example — computing the hash for "a":
The Russian letter а in UTF-8 is two bytes: `0xD0` (208) and `0xB0` (176).

```
hash = 0
hash = 0 * 31 + 208 = 208
hash = 208 * 31 + 176 = 6448 + 176 = 6624
result = 6624 % 127 = 20
```

So а goes to **bucket 20**. This matches the program output: The program confirms this: `Found: "а"(count: 1, bucket: 20)`
Similarly, б is `0xD0 0xB1`: hash $= 208 \times 31 + 177 = 6625$, $6625 \bmod 127 = 21$. в (`0xD0 0xB2`) gives bucket 22, г gives 23, д gives 24, е gives 25. These are sequential because the UTF-8 codes of consecutive Russian letters differ by exactly 1, so their hashes differ by 1. This is visible in the output:

```
=== Hash Table Dictionary ===
  [21] б(1)
  [22] в(1)
  [23] г(1)
  [24] д(1)
  [25] е(1)
  [52] ови(1)
```

No collisions here because each word maps to a different bucket.

5.6 Hash Table Operations Walkthrough

Insert “ови”:

The code converts “Ови” to lowercase “ови”, computes the hash (bucket 52), checks if the word exists in bucket 52’s chain (it does not), and creates a new node at the head:

```
void HashTable::insert(const string& word) {
    unsigned int index = hashFunction(word); // "ovi" -> 52
    // Walk the chain to check for duplicates
    HashNode* current = table[52]; // table[52] is nullptr (empty)
    // Word not found, so create new node:
    HashNode* newNode = new HashNode();
    newNode->word = word; newNode->count = 1;
    newNode->next = table[52]; // nullptr
    table[52] = newNode; // bucket 52 now points to "ovi"
}
```

Search “a” (found):

`Found: "a"(count: 1, bucket: 20)`

The code computes hash(a) = 20, walks bucket 20’s chain, finds “a”, and prints the result.

Delete “a” from hash table:

`Deleted "a"from bucket 20`

The code finds “a” in bucket 20 and removes it from the linked list:

```
bool HashTable::remove(const string& word) {
    unsigned int index = hashFunction(word); // "a" -> 20
    HashNode* current = table[20];
    HashNode* prev = nullptr;
    // current->word == "a" -> match found!
    // prev is nullptr, so this is the head:
    table[20] = current->next; // bucket 20 becomes empty
    delete current;            // free memory
    return true;
}
```

After deletion, the hash table has 5 words left (б at 21, в at 22, г at 23, д at 24, е at 25, and ови at 52). а is gone from the hash table but still exists in the RB-tree (they are independent data structures).

5.7 Input Validation

The program validates all user input:

```
Choice: v
Error: please enter a number (0-14).

Choice: 5d
Error: please enter a number (0-14).
```

This is handled in `main.cpp` by checking every character of the input:

```
bool validNumber = true;
for (size_t i = 0; i < choiceStr.size(); i++) {
    if (!isdigit((unsigned char)choiceStr[i])) {
        validNumber = false;
        break;
    }
}
```

When adding words, only Russian letters are accepted. The function `isAllRussian()` checks every byte pair in the UTF-8 string:

```
if (!isAllRussian(input)) {
    cout << "Error: only Russian letters are allowed." << endl;
    break;
}
```

5.8 Summary of All Operations Tested

Option	Operation	Verified
1	Load default text (Pushkin poem)	Yes
2	Load text from file (text1.txt)	Yes
3	Add word to RB-tree	Yes (а,б,в,г,д,е)
4	Search word in RB-tree	Yes (found в)
5	Delete word from RB-tree	Yes (б,в,г,д,е,а)
6	Print sorted dictionary (RB)	Yes
7	Print tree structure (RB)	Yes
8	Clear RB-tree	Yes
9	Add word to hash table	Yes (ови)
10	Search word in hash table	Yes (a at bucket 20)
11	Delete word from hash table	Yes (a from bucket 20)
12	Print hash table	Yes (6 words, no collisions)
13	Print hash table statistics	Yes
14	Clear hash table	Yes
0	Exit	Yes

Таблица 2. Summary of all tested operations

Список литературы

[1] Habr — Red-black trees. Electronic resource — URL: https://habr.com/ru/articles/330644/ (Accessed: 20.05.2026)

[2] Cormen T.H. et al. Introduction to Algorithms. — MIT Press, 2009. — 1312 p.

[3] “Telematics” section / electronic resource. — URL: https://tema.spbstu.ru/tgraph/ (Accessed: 28.05.2026).